Imágenes

Imágenes
- Intro
- Etiqueta `<img>`
- Estilizado básico de imágenes
- Etiquetas `<figure>` y `<figcaption>`
- Adecuación del objeto en el elemento
  - object-fit
  - object-position
  - image-orientation
- Filtros
- Recortado de cajas
- Imágenes responsive
- Fuentes icónicas
- Iconos SVG
- Máscaras
- Referencias

Intro

Las imágenes son uno de los elementos más importantes de las páginas web. De hecho, prácticamente todas las páginas web contienen alguna imagen y la mayoría incluyen decenas de imágenes. Dentro de las imágenes que se pueden incluir en una página HTML se deben distinguir dos tipos: **las imágenes de contenido y las imágenes de adorno.**

Las imágenes de contenido son las que proporcionan información y complementan la información textual. Las imágenes de adorno son las que se utilizan para hacer bordes redondeados, para mostrar pequeños iconos en las listas de elementos, para mostrar fondos de página, etc. Las imágenes de contenido se incluyen directamente en el código HTML mediante la etiqueta `<img>` y las imágenes de adorno no se deberían incluir en el código HTML, sino que deberían emplearse hojas de estilos CSS para mostrarlas.

Etiqueta `<img>`

La etiqueta `<img>` puede contener los siguientes atributos básicos:

- `src="url"`: Indica la URL de la imagen que se desea mostrar. Puede ser absoluta o relativa. Es un atributo obligatorio.
- `alt="texto"`: Descripción textual corta de la imagen. Necesario para personas con discapacidad visual. **Es obligatorio.**
- `longdesc="url"`: Indica una URL en la que puede encontrarse una descripción más detallada de la imagen. Sólo se usa cuando la descripción textual es de más de 1024 caracteres.

- `height="unidad_de_medida"`: Indica la altura con la que se debe mostrar la imagen (no es obligatorio que coincida con la altura original de la imagen). Se puede indicar también en porcentaje respecto a la anchura del padre. Si el valor no tiene unidad se sobreintendiende `px`.
- `width="unidad_de_medida"`: Indica la anchura con la que se debe mostrar la imagen (no es obligatorio que coincida con la anchura original de la imagen).
  Veamos un ejemplo:

```
<img src="/imagenes/foto1.jpg" alt="Fotografía de un paisaje" />
```

Se puede indicar también en porcentaje respecto a al altura del padre. Si el valor no tiene unidad se sobreintendiende `px`.

El estándar de HTML no ofrece una lista de formatos de imagen soportados, de modo que cada navegador soporta diferentes conjuntos de formatos, aunque los más habituales son soportados por todos los navegadores: **JPEG, GIF, GIF animado, PNG, APNG, SVG, BMP, BMP ICO, PNG ICO.**

En principio, la anchura y la altura con la que se muestra una imagen es parte de su aspecto gráfico, por lo que debería ser propio de CSS y no de HTML. Sin embargo, en algunas ocasiones no es práctico establecer la anchura y altura de todas las imágenes de contenidos mediante CSS debido a la sobrecarga de estilos diferentes que produciría. En estos casos puede suele ser más sencillo establecer la anchura y/o altura en la etiqueta `<img>` correspondiente.

La **anchura/altura con la que se muestra una imagen no tiene que coincidir obligatoriamente con la anchura/altura real de la imagen**. Sin embargo, cuando estos valores no coinciden, las imágenes se muestran deformadas y el aspecto final es muy desagradable. **Si solamente se establece el valor de una de las dos propiedades (altura o anchura), el valor de la otra propiedad es calculado automáticamente manteniendo la proporción de la imagen, haciendo que la imagen no se deforme.**

Estilizado básico de imágenes

Si queremos que una **imagen se adecúe al tamaño de su contenedor, deberemos indicar para ella que ocupe todo el ancho de su contenedor y que su altura se calcule automáticamente. En algunas ocasiones, si el contenedor es demasiado grande y la imagen no tiene mucho resolución es mejor indicar que la anchura máxima de la imagen sea del 100%,** de esta manera la imagen nunca sobresaldrá de su contenedor, pero cuando el contenedor sea muy grande la imagen no se agrandará tanto como para que se pixele:

```
img {
 width: 100%; /* max-width: 100% */
 height: auto;
}
```

Otro aspecto que debemos tener en cuenta es que, dado que la etiqueta `<img>` crea por defecto un elemento en línea, al incluirlo dentro de un contenedor se deja un pequeño margen inferior. Existen dos soluciones posibles:

- Establecer el `line-height` del contenedor a 0.
- Convertir la imagen en un elemento en bloque con `display: block`.

Veamos un ejemplo:

See the Pen Margen inferior en img by Pedro Joya (@pedrojoya) on CodePen.

```
<div class="contenedor">
  <img src="https://s3-us-west-2.amazonaws.com/s.cdpn.io/12005/harbour1.jpg"
alt="Bristol harbour">
</div>
<div class="contenedor solucion1">
  <img src="https://s3-us-west-2.amazonaws.com/s.cdpn.io/12005/harbour1.jpg"
alt="Bristol harbour">
</div>
<div class="contenedor">
  <img class="solucion2" src="https://s3-us-west-
2.amazonaws.com/s.cdpn.io/12005/harbour1.jpg" alt="Bristol harbour">
</div>
```

```
.contenedor {
  border: 1px solid black;
  background-color: beige;
  margin-top: 1em;
}
.contenedor img {
  height: 200px;
}
.solucion1 {
  line-height: 0;
}
.solucion2 {
  display: block;
}
```

> Demo de estilizado básico de imágenes

Por último, veamos un ejemplo con algunas opciones a la hora de mostrar texto junto a una imagen, como si de una tarjeta se tratase (sin flotar):

See the Pen img y texto by Pedro Joya (@pedrojoya) on CodePen.

```html
<div class="contenedor contenedor1">
  <img src="https://s3-us-west-2.amazonaws.com/s.cdpn.io/12005/harbour1.jpg"
alt="Bristol harbour">
  <span>Válido para una línea de texto</span>
</div>
<div class="contenedor contenedor2">
  <img src="https://s3-us-west-2.amazonaws.com/s.cdpn.io/12005/harbour1.jpg"
alt="Bristol harbour">
  <p>Lorem ipsum dolor, sit amet consectetur adipisicing elit. Velit tempora ad
ut error exercitationem. Illum quo quia molestias numquam eaque. Temporibus
molestiae similique nostrum cumque. Hic porro possimus dignissimos quod? Lorem
ipsum dolor sit amet consectetur adipisicing elit. Inventore sapiente ea,
nesciunt distinctio obcaecati quia maxime similique. Sed culpa velit, alias illo
amet minus ex. Cumque inventore tempora ea fuga! Lorem ipsum dolor sit amet
consectetur adipisicing elit. Odio, eos excepturi sed tempore dolore sit cum
ratione, odit praesentium asperiores vero ut? Ipsa, quis alias! Temporibus fugiat
nulla consequatur animi.fdsafasdf dsf d fad fda fa asf adfa d</p>
</div>
```

```css
.contenedor {
  border: 1px solid black;
  background-color: beige;
  margin-top: 1em;
}
.contenedor img {
  height: 200px;
}
.contenedor1 {
}
.contenedor1 img {
  vertical-align: middle;
}
.contenedor1 span {
  vertical-align: middle;
  padding: .5em;
}
.contenedor2 {
  display: table;
}
.contenedor2 img {
  display: table-cell;
}
.contenedor2 p {
  display: table-cell;
  vertical-align: top;
  padding: 1em;
  line-height: 1.5rem;
}
```

> Demo de texto junto a imagen

Etiquetas `<figure>` y `<figcaption>`

El elemento HTML `<figure>` representa contenido independiente, a menudo con un título. Por lo general, dicho **contenido se trata de una imagen, una ilustración, un diagrama, un fragmento de código,** o un esquema al que se hace referencia en el texto principal, pero que se puede mover a otra página o a un apéndice sin que afecte al flujo principal.

La etiqueta `<figure>` es una de las etiquetas semánticas añadidas por HTML 5, al igual que por ejemplo `<header>`, `<footer>`, etc, y al igual que éstas se trata de un elemento en bloque (como si de una etiqueta `<div>` se tratara).

Por su parte, el elemento HTML `<figcaption>` representa un subtítulo o leyenda asociado con una imagen o ilustración descrita por el resto de los datos asociados con el elemento `<figure>` padre de `<figcaption>`. Este elemento es opcional; y si no se especifica, el elemento padre `<figure>` no tendrá leyenda asociada.
Veamos un ejemplo de uso de estos dos elementos asociados a una imagen de contenido:

```
<figure>
  <img src="/imagenes/foto1.jpg" alt="Vista nocturna de la Alhambra" />
  <figcaption>Fig 1. - Vista nocturna de la Alhambra</figcaption>
</figure>
```

Adecuación del objeto en el elemento

Cuando usamos etiquetas que contendrán objetos, como por ejemplo `<img>`, que incluirá un objeto imagen, debemos tener en cuenta que puede que el tamaño del objeto imagen no coincida con el tamaño asignado mediante HTML o CSS al elemento que lo contendrá.

De **hecho el objeto tendrá unas dimensiones intrínsecas: su altura intrínseca, su achura intrínseca y su relación de aspecto intrínseca (la relación entre la anchura y la altura).** Estas dimensiones intrínsecas representan al tamaño preferido o natural del objeto en sí mismo, es decir, no son función del contexto en el que se utiliza el objeto. Algunos objetos pueden que no tengan definidas algunas de estas dimensiones intrínsecas, por ejemplo las imágenes SVG diseñadas a escala, que sólo proporcionan su relación de aspecto.

Por otra parte, **las dimensiones del elemento donde se mostrará el objeto son dependientes del contexto** en donde se encuentre, por ejemplo el tamaño de su elemento padre, y también del valor de determinadas propiedades CSS que hayamos especificado, como por ejemplo `width`, `height`, `min-width`. etc.

object-fit

Por tanto el navegador debe mostrar el objeto dentro de la caja correspondiente a dicho elemento, lo que puede conllevar el redimensionado del objeto. Para controlar este proceso, podemos hacer uso de la `object-fit`, que establece como se debe mostrar el objeto dentro de su caja.

Propiedad	object-fit
Valores	fill | contain | cover | none | scale-down
Se aplica a	Elementos que contienen objetos

Propiedad	object-fit
Valor inicial	fill
Descripción	Establece cómo se adecúa el objeto dentro del elemento

El significado de los valores es el siguiente:

- `fill`: **El objeto se redimensiona para llenar la caja del elemento.** Las dimensiones del objeto se ajustan al valor correspondiente del elemento , si es que han sido especificados, incluso perdiendo su relación de aspecto. El objeto ocupará siempre todo el elemento, pero puede quedar deformado. Es el valor por defecto.
- `contain`: **El objeto se redimensiona manteniendo su relación de aspecto de manera que entre entero dentro del elemento sin deformarlo. Puede no ocupar todo el ancho o alto del elemento.**
- `cover`**: El objeto se redimensiona manteniendo su relación de aspecto para cubrir toda la superficie de la caja contenedora**, realizando un recorte en el objeto de la parte sobrante que no entra en el elemento. **El elemento se cubre por completo, pero el objeto puede no mostrarse entero.**
- `none`: El objeto NO se redimensiona. El tamaño con el que se mostrará el objeto dependerá del tipo de objeto del que se trate (y del algoritmo usado por él).
- `scale-down`: El objeto se dimensiona al tamaño más pequeño entre el tamaño original y el tamaño resultante de redimensionarlo manteniendo su relación de aspecto apra colocarlo dentro de su caja, es decir el de menor tamaño entre aplicar los valores `none` o `contain`.

Si queremos forzar a que el elemento tome las dimensiones del objeto debemos usar para el elemento las propiedades `max-width` y `max-height` en vez de `width` y `height`.

object-position

Si aplicamos a la propiedad `object-fit` un valor distinto a `fill`, el objeto se muestra por defecto centrado en el elemento, aunque podemos variar este comportamiento con la propiedad `object-position`, que determina la posición del objeto dentro del elemento.

Propiedad	object-position
Valores	(unidad de medida | porcentaje) {1, 4}
Se aplica a	Elementos que contienen objetos
Valor inicial	50% 50%
Descripción	Establece la posición del objeto dentro del elemento

Recibe de uno a cuatro valores que representan la posición de objeto respecto a los bordes de la caja del elemento. Los valores porcentuales se refieren respecto a la anchura y altura de la caja.

Veamos una demostración (by Kseso):

See the Pen Propiedades object-fit y object-position by Pedro Joya (@pedrojoya) on CodePen.

```
<img class='uno' src='https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg' alt="Foto de Marilyn
Monroe" />
<img class='dos' src='https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg' alt="Foto de Marilyn
Monroe" />
<img class='tres' src='https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg' alt="Foto de Marilyn
Monroe" />
```

```
img {
  background: #0ebeff;
  /* Comportamiento por defecto */
  object-fit: fill;
  object-position: 50% 50%;
  /** Realiza cambios en las propiedades object-fill y object-position y
comprueba cual es el efecto **/
  box-shadow: 0 0 4px 1px rgba(0, 0, 0, .4);
  vertical-align: middle;
}
.uno {
  width: 25vw;
  height: 100px;
}
.dos {
  width: 25vw;
  height: 400px;
}
.tres {
  max-width: 50vw;
  max-height: 500px;
}
```

> Demo de propiedad object-position (by Kseso)

En la siguiente web tiene una demostración interactiva:

> Demo interactiva de la propiedad object-position

image-orientation

CSS también define la propiedad `image-orientation`, que permite girar el objeto una serie de grados al situarlo dentro del elemento. Sin embargo esta propiedad sólo es soportada actualmente por Firefox.

Filtros

La propiedad `filter` dota de efectos como el desenfoque o cambio de color en la representación de un elemento antes de que se muestre el elemento. Los filtros se utilizan comúnmente para ajustar la representación de una imagen.

Propiedad	**filter**
Valores	none | Uno o más
Se aplica a	Todos los elementos
Valor inicial	none
Descripción	Aplica uno o más filtros de visualización al elemento

Los filtros se pueden proporcionar a través de la url de un elemento `<filter>` incluido dentro de un elemento `<svg>` local, o a través de alguna función de filtro.

Veamos los filtros disponibles:

- `blur(radio_desenfoque)`: Aplica un desenfoque Gaussiano a la imagen. El valor de 'radio' define el valor de la desviación estándar de la función de desenfoque Gaussiano o el número de píxeles que se mezclan entre sí, por lo que un valor mayor creará un mayor desenfoque.
- `brightness(%brillo)`: Se aplica una multiplicación lineal a la imagen, haciendo que parezca más o menos brillante. El valor del brillo de la imagen original corresponde al valor 100%, que es el valor por defecto. Así, un valor de 0% convertir la imagen completamente a negro y un valor mayor que 100% aumentará el brillo de la imagen.
- `contrast(%contraste)`: Ajusta el contraste de la imagen. El valor del contraste de la imagen original corresponde al valor 100%, que es el valor por defecto. Así, un valor de 0% hace que la imagen sea completamente negra y valor mayor de 100% proporcionan imágenes con mayor contraste.
- `saturate(%saturación)`: Ajusta la saturación de la imagen. El valor de la saturación de la imagen original corresponde al valor 100%, que es el valor por defecto. Así, un valor de 0% hace que la imagen aparezca totalmente insaturada y valor mayor de 100% proporcionan imágenes con mayor saturación.
- `grayscale(%conversion)`: Convierte la imagen a escala de grises. Un valor de 100% es mostrada completamente en escala de grises. Un valor de 0% no modifica la imagen (valor por defecto).
- `sepia(%conversion)`: Convierte la imagen a color sepia. Un valor de 100% es mostrada completamente en sepia. Un valor de 0% no modifica la imagen (valor por defecto).
- `invert(%inversion)`: Invierte los colores de la imagen. Un valor de 100% invierte completamente los colores. Un valor de 0% no modifica la imagen (valor por defecto).
- `hue-rotate(90deg)`: Rota la tonalidad de los colores los grados indicados. Un valor de 0 grados no modifica la imagen (valor por defecto). Si valor proporcionado es mayor de 360, se sigue girando.

- `drop-shadow()`: Aplica una sombra a la imagen. Recibe los mismos valores que la propiedad `box-shadow`, excepto `inset`, que no está permitido. Esta función es similar a utilizar la propiedad `box-shadow`, aunque algunos navegadores proporcionan aceleración por hardware si usamos este filtro en vez de la propiedad `box-shadow`, mejorando el rendimiento.
- `opacity(%opacidad)`: Establece la opacidad de la imagen. El valor 100% (valor por defecto) muestra la imagen sin transparencia. El valor 0% hace a la imagen completamente transparente.

Podemos combinar las funciones anteriores dentro del valor de la propiedad `filter`, tan sólo tendremos que separar las llamadas a las funciones mediante espacio en blanco.

Aparte de los filtros disponibles a través de funciones, podemos usar como valor de la propiedad `filter` la URL de un elemento `<filter>` contenido dentro de un elemento `<svg>` local. El trabajo con los filtros SVG es algo más complejo.

Veamos unas demos interactivas con los distintos valores de filtro:

> [Demo interactiva 1 de las propiedad filter]
>
> [Demo interactiva 2 de las propiedad filter]

A continuación se muestra un ejemplo:

See the Pen [Propiedad filter] by Pedro Joya ([@pedrojoya]) on [CodePen].

```
<!-- Original -->
<figure>
  <img src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>Original</figcaption>
</figure>
<!-- Desenfoque -->
<figure>
  <img class="blur" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>blur(4px)</figcaption>
</figure>
<!-- Brillo -->
<figure>
  <img class="brightness" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
 <figcaption>brightness(50%)</figcaption>
</figure>
<!-- Contraste -->
<figure>
  <img class="contrast" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>contrast(150%)</figcaption>
</figure>
<!-- Escala de grises -->
<figure>
  <img class="grayscale" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>grayscale(60%);</figcaption>
</figure>
```

```html
<!-- Inversión de colores -->
<figure>
  <img class="invert" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>invert(100%)</figcaption>
</figure>
<!-- Saturación -->
<figure>
  <img class="saturate" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>saturate(200%)</figcaption>
</figure>
<!-- Sepia -->
<figure>
  <img class="sepia" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>sepia(100%)</figcaption>
</figure>
<!-- Rotación de tonalidad -->
<figure>
  <img class="hue-rotate" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>hue-rotate(180deg)</figcaption>
</figure>
<!-- Sombra -->
<figure>
  <img class="drop-shadow" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>drop-shadow(5px 10px 8px)</figcaption>
</figure>
<!-- Transparencia -->
<figure>
  <img class="opacity" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>opacity(50%)</figcaption>
</figure>
<!-- Varios filtros -->
<figure>
  <img class="varios" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>Varios filtros</figcaption>
</figure>
<!-- Con URL -->
<figure>
  <img class="con-url" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
  <figcaption>url(#desenfoque)</figcaption>
</figure>
<!-- Definición de filtros -->
<svg>
  <filter id="desenfoque">
    <feGaussianBlur in="SourceGraphic" stdDeviation="20"/>
  </filter>
</svg>
```

```
html {
  text-align: center;
}
figure, figcaption {
  padding-top: 1rem;
}
.avatar {
  margin-top: 50px;
}
img {
  margin: 10px;
  width: 250px;
}
.blur { filter: blur(4px); }
.brightness { filter: brightness(50%); }
.contrast { filter: contrast(150%); }
.grayscale { filter: grayscale(60%); }
.invert { filter: invert(100%); }
.saturate { filter: saturate(200%); }
.sepia { filter: sepia(100%); }
.hue-rotate { filter: hue-rotate(180deg); }
.drop-shadow { filter: drop-shadow(5px 10px 8px); }
.opacity { filter: opacity(50%); }
.varios { filter: contrast(120%) brightness(90%) saturate(150%) drop-shadow(5px
5px 10px); }
.con-url { filter: url(#desenfoque); }
```

> Demo de propiedad filter

Por su parte, la propiedad `backdrop-filter` permite aplicar filtros al área situada detrás de un elemento. Para que vean estos efectos es necesario que el elemento sea parcialmente transparente.

Propiedad	backdrop-filter
Valores	none | Uno o más
Se aplica a	Todos los elementos
Valor inicial	none
Descripción	Aplica uno o más filtros de visualización a lo que haya detrás de un elemento

Los posibles filtros son: `blur()`, `brightness()`, `contrast()`, `drop-shadow()`, `grayscale()`, `hue-rotate()`, `invert()`, `opacity()`, `saturate()`, o `sepia()`, con los mismo significados que en la propiedad `filter`.

Actualmente sólo está disponible por ahora en el navegador Safari usando el prejijo `-webkit-`.

> Para saber más

Recortado de cajas

Tradicionalmente todos los elementos han sido mostrados con forma rectangular, es decir, en cajas con cuatro esquinas con un ángulo de 90 grados. Con el uso de `border-radius` **se "enmascaran" las esquinas para que aparezcan redondeadas.**

Para poder solucionar este problemas, CSS3 incluye la propiedad `clip-path`, que permite aplicar patrones de recorte a los elementos, de manera que estos luzcan con formas distintas a la rectangular.

Propiedad	**clip-path**
Valores	none | <url_origen_recorte> | [<forma-básica> | <caja_referencia>]
Se aplica a	Todos los elementos
Valor inicial	none
Descripción	Establece el modo de recorte del elemento

Veamos los valores posibles:

- `<url_origen_recorte>`: Hace referencia a la URL del elemento que va a ser usado para recortar la imagen. Dicho elemento debe ser un contenido dentro de un elemento . Actualmente, los navegadores sólo permiten trabajar en esta propiedad con URL locales, por lo que deberemos definir nuestro elemento de forma local y asignarle una id al elemento `<clipPath>` correspondiente para que podamos indicar su URL relativa haciendo uso de `#id`. Esta posibilidad es compatible actualmente tanto en Firefox como en Chrome.
- `<forma_basica>`: Como alternativa a especificar la URL, podemos usar una forma básica para indicar el elemento que debe ser usado para recortar la imagen. Esta posibilidad sólo está presente por ahora en Chrome. Para definir la forma básica se puede usar una de las siguientes funciones:
    - `circle(<radio> at <posición>)`: El recorte se realizará con un círculo del radio indicado situado en la posición indicada. Por ejemplo:

```
/* Recorte con círculo con radio del 50% situado en el centro */
img {
  clip-path: circle(50% at 50% 50%);
}
```

- `ellipse(<radio1> <radio2> at <posición>)`: El recorte se realizará con una elipse de los radios indicados en la posición indicada.. Por ejemplo:

```
/* Recorte en forma de elipse */
img {
  clip-path: ellipse(50% 35% at 50% 50%);
}
```

- `inset(<long1> <long2> <long3> <long4> round <radio_esquinas>)`: El recorte se realizará mediante un rectángulo interior situado a la longitud indicada respecto a cada lado y con las esquinas redondeadas con el radio indicado. Por ejemplo:

```
/* Recorte mediante un rectángulo interior con una distancia del 10%
respecto al borde de la caja y las esquinas redondeadas con un radio de
10px */
img {
  clip-path: inset(10% round 10px);
}
```

- `polygon(puntos)`: El recorte se realizará siguiente el polígono indicado por los puntos. Por ejemplo:

```
/* Recorte mediante el polígono indicado (un triángulo en este caso) */
img {
  clip-path: polygon(50% 0%, 0% 100%, 100% 100%);
}
```

Veamos una demostración:

See the Pen [Propiedad clip-path](#) by Pedro Joya (@pedrojoya) on CodePen.

```
  <!-- Círculo -->
  <figure>
    <img class="circulo" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>circle(50%)</figcaption>
  </figure>
  <!-- Elipse -->
  <figure>
    <img class="elipse" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>ellipse(80% 40%)</figcaption>
  </figure>
  <!-- Rectángulo interno -->
  <figure>
    <img class="rectangulo" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>inset(10% round 8px)</figcaption>
  </figure>
```

```
  <!-- Triángulo -->
  <figureº
    <img class="triangulo" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>polygon(50% 0%, 0% 100%, 100% 100%);</figcaption>
  </figure>
  <!-- Diamante -->
  <figure>
    <img class="diamante" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>polygon(50% 0, 100% 50%, 50% 100%, 0 50%)</figcaption>
  </figure>
  <!-- Pentágono -->
  <figure>
    <img class="pentagono" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>polygon(50% 0, 100% 38%, 80% 100%, 20% 100%, 0 38%)</figcaption>
  </figure>
  <!-- Hexágono -->
  <figure>
    <img class="hexagono" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>polygon(50% 0, 95% 25%, 95% 75%, 50% 100%, 5% 75%, 5% 25%)
</figcaption>
  </figure>
  <!-- Octógono -->
  <figure>
    <img class="octogono" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>polygon(0 50%, 15% 15%, 50% 0, 85% 15%, 100% 50%, 85% 85%, 50%
100%, 15% 85%);</figcaption>
  </figure>
  <!-- Corazón -->
  <figure>
    <img class="corazon" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>corazón</figcaption>
  </figure>
  <!-- Letras -->
  <figure>
    <img class="letras" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>Letras (sólo Firefox)</figcaption>
  </figure>
  <!-- Mano -->
  <figure>
    <img class="mano" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>Mano</figcaption>
  </figure>
    <!-- Máscara -->
  <figure>
    <img class="mascara" src="https://3.bp.blogspot.com/-
l4cAL1OFLYo/VDfLDCCk12I/AAAAAAAAKJw/Zd4wdhoAetc/s500/mr.jpg" alt="" />
    <figcaption>Máscara (sólo Firefox)</figcaption>
  </figure>
```

```
  <!-- Definición de zonas de recorte (para Firefox) -->
  <svg>
  <defs>
    <!-- Círculo -->
    <clipPath id="circle">
      <circle cx="150" cy="150" r="150"/>
    </clipPath>
    <!-- Elipse -->
    <clipPath id="ellipse">
      <ellipse cx="150" cy="150" rx="150" ry="120"/>
    </clipPath>
    <!-- Rectángulo -->
    <clipPath id="rect">
      <rect x="10" y="10" rx="8" ry="8" width="280" height="240" />
    </clipPath>
    <!-- Triángulo -->
    <clipPath id="triangle" clipPathUnits="objectBoundingBox">
      <polygon points=".5,0 0,1 1,1" />
    </clipPath>
    <!-- Diamante -->
    <clipPath id="diamond" clipPathUnits="objectBoundingBox">
      <polygon points=".5,0 1,.5 .5,1 0,.5" />
    </clipPath>
    <!-- Pentágono -->
    <clipPath id="pentagon" clipPathUnits="objectBoundingBox">
      <polygon points=".5,0 1,.38 .8,1 .2,1 0,.38" />
    </clipPath>
    <!-- Hexágono -->
    <clipPath id="hexagon" clipPathUnits="objectBoundingBox">
      <polygon points=".5,0 .95,.25 .95,.75 .5,1 .05,.75 .05,.25" />
    </clipPath>
    <!-- Octágono -->
    <clipPath id="octogon" clipPathUnits="objectBoundingBox">
      <polygon points="0,.5 .15,.15 .5,0 .85,.15 1,.5 .85,.85 .5,1 .15,.85" />
    </clipPath>
    <!-- Corazón -->
    <clipPath id="heart">
            <path id="heart" d="M248.078,5.883c-36.691-14.739-77.771-0.839-
98.517,31.125C128.817,5.044,87.735-8.856,51.043,5.883 C9.354,22.632-
10.863,70.009,5.887,111.696c16.06,39.98,143.314,139.607,143.314,139.607l0.359,0.2
8l0.36-0.28 c0,0,127.251-99.627,143.314-
139.607C309.986,70.009,289.768,22.632,248.078,5.883z"/>
        </clipPath>
    <!-- Texto -->
    <clipPath id="cliptext">
      <text x="200" y="150" transform="rotate(30 20,40)" font-size="180px"
style="font-size: 180px; font-weight: 800;" text-anchor="middle">MM</text>
    </clipPath>
    <!-- Mano alzada -->
    <clipPath id="hand">
```

```
      <polygon points="42 181,44 183,45 185,48 188,48 189,49 190,50 190,50 192,51
193,51 212,53 214,54 216,55 217,58 218,61 220,63 222,66 224,74 228,79 230,91
236,96 238,98 240,100 241,102 241,106 243,112 247,114 247,116 249,117 249,120
252,124 254,125 255,127 255,128 256,133 259,137 263,140 265,141 266,142 266,144
267,148 271,151 272,153 273,156 275,158 276,160 276,163 278,188 278,197 275,214
275,217 277,224 280,232 282,242 284,251 285,291 285,297 284,302 284,309 283,313
282,317 280,320 278,326 276,330 274,339 271,343 269,348 268,352 266,355 263,360
261,370 255,376 254,380 254,382 252,384 251,385 250,387 246,390 244,399 235,402
233,406 228,412 219,413 217,413 215,415 211,416 210,418 207,420 203,423 199,425
195,426 188,427 185,429 182,430 178,430 158,428 150,428 147,426 141,424 136,423
133,422 131,422 129,421 127,420 124,419 119,415 107,414 103,411 94,409 91,406
89,404 88,384 88,379 89,368 89,364 90,351 90,351 92,350 93,348 94,344 98,342
101,340 103,336 109,334 111,332 114,330 118,328 120,327 123,325 126,324 130,320
142,320 145,318 148,314 160,310 166,307 169,305 171,301 173,295 173,293 171,291
170,290 170,289 169,287 169,286 168,285 168,284 167,284 166,280 154,274 145,273
140,270 137,270 134,269 132,268 131,268 128,267 127,267 126,264 120,261 115,259
113,256 109,252 105,249 102,246 98,242 95,240 93,238 92,229 92,228 93,224 95,222
97,219 99,217 100,214 102,212 103,209 104,204 104,202 106,201 108,201 109,199
113,196 122,196 127,193 131,192 136,191 139,190 144,189 148,189 152,188 155,188
159,186 163,178 175,175 178,173 179,167 179,165 175,161 171,160 169,159 168,158
166,156 165,156 163,155 161,153 159,152 157,152 155,150 149,149 145,148 142,148
139,147 136,146 132,144 128,143 124,143 122,142 120,142 118,141 118,141 116,140
114,138 111,135 109,133 107,131 104,129 103,127 101,124 99,121 98,101 98,98 99,94
100,90 102,87 104,82 106,76 110,72 112,66 116,63 120,62 123,60 126,59 130,57
133,53 143,49 155,48 159,47 164,45 169,45 182">
    </polygon>
  </clipPath>
    <!-- Máscara -->
    <linearGradient id="gradient" x1="0" y1="0%" x2 ="0" y2="100%">
      <stop stop-color="white" offset="0"/>
      <stop stop-color="#444" offset="1"/>
    </linearGradient>
    <mask id="masking" maskUnits="objectBoundingBox"
maskContentUnits="objectBoundingBox">
      <rect y="0" width="1" height="1" fill="url(#gradient)" />
    </mask>
</defs>
</svg>
```

```
html {
  text-align: center;
}
.avatar {
  margin-top: 50px;
}
img {
  margin: 10px;
}
.circulo {
  -webkit-clip-path: circle(50%);
  clip-path: url(#circle);
}
.elipse {
  -webkit-clip-path: ellipse(80% 40%);
  clip-path: url(#ellipse);
```

```
}
.rectangulo {
  -webkit-clip-path: inset(10% round 8px);
  clip-path: url(#rect);
}
.triangulo {
  -webkit-clip-path: polygon(50% 0%, 0% 100%, 100% 100%);
  clip-path: url(#triangle);
}
.diamante {
  -webkit-clip-path: polygon(50% 0, 100% 50%, 50% 100%, 0 50%);
  clip-path: url(#diamond);
}
.pentagono {
  -webkit-clip-path: polygon(50% 0, 100% 38%, 80% 100%, 20% 100%, 0 38%);
  clip-path: url(#pentagon);
}
.hexagono {
  -webkit-clip-path: polygon(50% 0, 95% 25%, 95% 75%, 50% 100%, 5% 75%, 5% 25%);
  clip-path: url(#hexagon);
}
.octogono {
  -webkit-clip-path: polygon(0 50%, 15% 15%, 50% 0, 85% 15%, 100% 50%, 85% 85%,
50% 100%, 15% 85%);
  clip-path: url(#octogon);
}
.corazon {
  width: 300px;
  -webkit-clip-path: url(#heart);
  clip-path: url(#heart);
}
.letras {
  width: 280px;
    height: 280px;
  -webkit-clip-path: url(#cliptext);
  clip-path: url(#cliptext);
}
.mano {
  -webkit-clip-path: url(#hand);
  clip-path: url(#hand);
}
.mascara {
  -webkit-mask: url(#masking);
  mask: url(#masking);
}
```

> Demo de propiedad clip-path

Afortunadamente, tenemos disponibles online algunos generadores para obtener de forma visual los valores para la propiedad `clip-path`:

> http://cssplant.com/clip-path-generator
>
> http://bennettfeely.com/clippy/

Imágenes responsive

Como hemos visto, es relativamente **sencillo hacer que una elemento `<img>` se adapte al tamaño de pantalla, bastará con establecer para dicho elemento la propiedad `width` a un valor porcentual y, opcionalmente, las propiedades `min-width` y/o `max-width` a un valor fijo.**

Incluso hemos visto cómo controlar la adaptación del objeto imagen dentro del elemento `<img>` gracias a las propiedades `object-fit` y `object-position`, con las que podemos establecer, respectivamente, el tipo de escalado de la imagen y su posición dentro del elemento.

Sin embargo estas técnicas distan de ser ideales en la realidad, porque **si queremos que la imagen se visualice bien en tamaños y resoluciones de pantalla muy dispares**, como por ejemplo un dispositivo móvil de baja resolución y una pantalla de escritorio de alta resolución, nuestra mejor opción sería proporcionar el objeto imagen en el tamaño más grande y en la mayor resolución que vaya a necesitar, y que sea escalado hacia un tamaño inferior cuando sea necesario. De esta forma la imagen se mostrará siempre con buena calidad independientemente del dispositivo. No obstante, dicha calidad tiene un precio: el "peso" de la imagen, es decir, el número de MB que tendrán que descargarse para mostrar la imagen, incluso aunque la imagen vaya a mostrarse finalmente en un tamaño mucho menor y con una resolución mucho menor.

Para solucionar este problema HTML5 incorporó el atributo `srcset` a la etiqueta `<img>`, que permite al desarrollador especificar distintas versiones de la imagen, indicando su ancho y otros factores, como la resolución, la orientación, etc., de manera que el navegador sea capaz de elegir la versión más óptima en cada momento.

El atributo `srcset` es una extensión del atributo `src`. Su valor corresponde a una lista de una o más cadenas separadas por comas indicando las distintas "versiones" de la imagen. Cada cadena describe las características de dicha "versión", haciendo uso de la sintaxis `url [ancho] [radio_densidad]`, donde:

- `url`: Indica la url del objeto imagen.
- `ancho`: Indica la anchura del objeto imagen. Se especifica con un número positivo seguido de la letra `w`. Es opcional.
- `ratio_densidad`: Indica el número de píxeles del dispositivo respecto al del CSS. Se especifica por un número positivo decimal seguido de la letra `x`. Se usa con pantallas retina como las del mac, que tienen un valor `2x`. Es opcional y su valor por defecto es `1x`.
  No es válido mezclar ancho y ratio_densidad en el mismo atributo `srcset`, ni repetir descriptores en varias cadenas.

Veamos un ejemplo:

```
<img
    srcset="imagenes/foto.jpg 200w, imagenes/foto-grande.jpg 400w, imagenes/foto-enorme.jpg 600w, imagenes/foto-2x.jpg 2x"
    src="imagenes/foto.jpg"/>
```

El navegador seleccionará automáticamente la "versión" más adecuada de la imagen atendiendo al tamaño del elemento y al ratio de densidad del dispositivo. Se recomienda siempre incluir también el atributo `src` con el valor tradicional, por si el navegador no soportara el atributo `srcset`.

Veamos una demostración (ver en Firefox, porque en Chrome hay un fallo de refresco, ya que una vez obtenida una versión de mayor tamaño no se actualizará a una de tamaño menor, aunque en el sentido inverso si funciona):

See the Pen Atributo srcset by Pedro Joya (@pedrojoya) on CodePen.

```
<!-- Probar mejor en Firefox (ve redimensionando la ventana) --->
<img src="https://dummyimage.com/200x200/D4B56A/fff.png&text=200"
     srcset="https://dummyimage.com/200x400/D4B56A/fff.png&text=200 200w,
             https://dummyimage.com/400x400/D4B56A/fff.png&text=400 400w,
             https://dummyimage.com/600x400/D4B56A/fff.png&text=600 600w,
             https://dummyimage.com/800x400/D4B56A/fff.png&text=800 800w,
             https://dummyimage.com/1000x400/D4B56A/fff.png&text=1000 1000w,
             https://dummyimage.com/1200x400/D4B56A/fff.png&text=1200 1200w,
             https://dummyimage.com/1400x400/D4B56A/fff.png&text=1400 1400w,
             https://dummyimage.com/1600x400/D4B56A/fff.png&text=1600 1600w"
     alt="foto"/>
```

```
body {
  display: flex;
  min-height: 100vh;
  justify-content: center;
  align-items: center;
}
```

> Demo de atributo srcset

Otro de los nuevos atributos introducidos por HTML5 para la etiqueta `<img>` es el atributo `sizes`**, que permite indicar al navegador la anchura que va a tener la caja correspondiente a una etiqueta** `<img>` **dependiendo de la anchura del *viewport*., con objeto de que el navegador puede elegir la "versión" óptima** de la imagen. Este atributo recibe una lista de una o más cadenas separadas por comas, cada una de las cuales consiste en:

- Una condición de anchura de *viewport* similar a las de las *media queries*, que sirve para indicar cuándo debe usarse el tamaño que se proporcionará a continuación. Esta condición debe omitirse en la última cadena, ya que ésta sólo incluirá el tamaño por defecto de la caja para a la etiqueta `<img>`
- Un valor de tamaño, correspondiente a la anchura de la caja para la etiqueta `<img>` cuando la anchura del *viewport* cumple con la condición especificada anteriormente.
  Veamos un ejemplo:

```
<img
    srcset="imagenes/foto.jpg 200w, imagenes/foto-grande.jpg 400w, imagenes/foto-enorme.jpg 600w"
    sizes="(max-width: 300px) 100vw, (max-width: 600px) 66vw, 33vw"
    src="images/foto.jpg"/>
```

En el ejemplo anterior estamos indicando que:

- **cuando la anchura del *viewport* sea como máximo de 300px el elemento** `<img>` **ocupará el 100% de la anchura del *viewport***

- **cuando la anchura del *viewport* sea como máximo de 600px el elemento `<img>` ocupará el 66% de la anchura del viewport**
- En el resto de casos el elemento `<img>` ocupará el 33% de la anchura del *viewport*

El navegador usará el valor de este atributo para determinar la imagen adecuada de entre las especificadas en el atributo `srcset`, siempre y cuando se haya indicado el ancho de las imágenes en dicho atributo. Si no ha especificado el atributo `srcset`, o no contiene imagenes para las que se haya definido un ancho, entonces el atributo `sizes` no tiene efecto.

Veamos una demostración (ver en Firefox, porque en Chrome hay un fallo de refresco, ya que una vez obtenida una versión de mayor tamaño no se actualizará a una de tamaño menor, aunque en el sentido inverso sí funciona):

See the Pen Atributo sizes y srcset by Pedro Joya (@pedrojoya) on CodePen.

```
<!-- Probar mejor en Firefox --->
<img
  sizes="(max-width: 300px) 100vw, (max-width: 600px) 66vw, 33vw"
  srcset="https://dummyimage.com/200x400/D4B56A/fff.png&text=200 200w,
          https://dummyimage.com/400x400/D4B56A/fff.png&text=400 400w,
          https://dummyimage.com/600x400/D4B56A/fff.png&text=600 600w,
          https://dummyimage.com/800x400/D4B56A/fff.png&text=800 800w,
          https://dummyimage.com/1000x400/D4B56A/fff.png&text=1000 1000w,
          https://dummyimage.com/1200x400/D4B56A/fff.png&text=1200 1200w,
          https://dummyimage.com/1400x400/D4B56A/fff.png&text=1400 1400w,
          https://dummyimage.com/1600x400/D4B56A/fff.png&text=1600 1600w"
  alt="foto">
```

```
body {
  display: flex;
  min-height: 100vh;
  justify-content: center;
  align-items: center;
}
```

> Demo del atributo sizes

El elemento `<picture>` es un contenedor que permite al desarrollador proporcionar múltiples "versiones" (orígenes) de una imagen, para que el navegador elija qué versión es la más idonea para mostrar en el elemento en base a una serie de criterios. Se trata pues de una extensión de los conceptos introducidos por los atributos `srcset` y `sizes`.
Un elemento `<picture>` debe tener obligatoriamente los siguientes hijos:

- Al menos un elemento `<source>`, en el que se especifique al menos una "versión" (origen) de la imagen.
- Un elemento `<img>`, en el que se defina el atributo `src`.
  Por ejemplo:

```
<picture>
  <source srcset="imagenes/foto.jpg 200w, imagenes/foto-grande.jpg 400w,
imagenes/foto-enorme.jpg 600w" />
  <img src="imagenes/foto.jpg" alt="Foto" />
</picture>
```

El elemento `<source>` representa, por su parte, un origen o fuente para la obtención de una o varias versiones de la imagen que se desea mostrar. Sus atributos principales son:

- `srcset`: Mismo significado que en el elemento `<img>`. Es obligatorio.
- `sizes`: Mismo significado que en el elemento `<img>`. Es opcional.
- `media`: Indica una condición de *media query* para la que se sugiere al navegador usar el `scrset` indicado. Es opcional. Por ejemplo:

  ```
  <picture>
    <source media="(min-width: 45rem)" srcset="imagenes/foto-grande.jpg">
    <source media="(min-width: 18rem)" srcset="imagenes/foto-mediana.jpg">
    <img src="imagenes/foto-pequeña.jpg" alt="Foto" />
  </picture>
  ```

  En el ejemplo anterior, se sugiere al navegador que se use la foto mediana si el *viewport* tiene una anchura mayor de `18rem`, la foto grande si es mayor de `45rem` y la foto pequeña en otro caso.

  Otro ejemplo característico es hacer que la "versión" de la imagen sea distinta dependiendo de si el dispositivo está en modo retrato o en modo apaisado. Por ejemplo:

  ```
  <picture>
      <source media="(orientation: landscape)" srcset="imagenes/foto-
  apaisada.jpg">
      <source media="(orientation: portrait)" srcset="imagenes/foto-
  retrato.jpg">
      <img src="imagenes/foto.jpg" alt="Foto" />
  </picture>
  ```

- `type`: Formato de archivo para el que se sugiere al navegador usar el `srcset` indicado. Es opcional. Por ejemplo:

  ```
  <picture>
    <source type="image/webp" srcset="imagenes/foto.webp">
  </picture>
  ```

  En el ejemplo anterior, los navegadores que soporten el formato de imagen `webp` mostrarán dicho archivo, el resto mostrarán el archivo `jpg`.
  Por otra parte tenemos disponibles algunos generadores online:

> Generador online para etiqueta `<picture>`

Fuentes icónicas

Cuando tenemos que mostrar iconos una de las opciones habituales es usar las conocidas como fuentes icónicas, que fuentes (tipos de letras) que, en lugar de incluir letras, incluyen iconos como si de letras se tratase. La ventajas principales es que estas fuentes son:

- Se tratan de grafismos vectoriales, por lo que no se deforman o pixelan al cambiarlas de tamaño.
- Podemos aplicar un color al icono con tan sólo cambiar el color de la fuente.

- Pesan mucho menos que las imágenes por lo que el cliente tendrá que descargar menos datos.
  Algunas de las fuentes icónicas más conocidas son FontAwesome, Themify, Octicons, o Fontastic.
  A modo de ejemplo, veamos como añadir una serie de iconos de FontAwesome a una web:

```
<head>
  <!-- Script de inclusión de la librería de iconos
  <script defer src="https://use.fontawesome.com/releases/v5.0.4/js/all.js">
</script>
</head>
<body>
  <!-- Icono de usuario con dos estilos diferentes -->
  <i class="fas fa-user"></i>
  <i class="far fa-user"></i>
  <!-- Icono de marca -->
  <i class="fab fa-github-square"></i>
</body>
```

Veamos una demostración de FontAwesome:

See the Pen Font Awesome CSS font by Pedro Joya (@pedrojoya) on CodePen.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Font Awesome CSS font</title>
  <link href="https://use.fontawesome.com/releases/v5.0.6/css/all.css"
rel="stylesheet">
  <link href="style.css" rel="stylesheet">
</head>
<body>
  <p>Redes sociales:</p>
  <p class="redes">
    <i class="fab fa-google-plus-square"></i>
    <i class="fab fa-facebook-square"></i>
    <i class="fab fa-twitter-square"></i>
    <i class="fab fa-instagram"></i>
  </p>
</body>
</html>
```

```
.fab { font-size: 40px; }
.redes i { margin-right: 1rem;}
.fa-google-plus-square { color: #d34836; }
.fa-facebook-square { color: #3a5998; }
.fa-twitter-square { color: #00aced; }
.fa-instagram { color: #8a3ab9; }
```

> Demo de FontAwesome

Iconos SVG

Debemos tener en cuenta que las fuentes icónicas no dejan de ser un "truco" (*hack*) para tratar los iconos como si se tratasen de caracteres de un tipo de letra específico. De hecho, su empleo presenta algunos problemas, como por ejemplo el hecho de que si, por el motivo que sea, no se puede mostrar el icono correspondiente el navegador simplemente mostrará un cuadrado negro. Además, debemos tener en cuenta que los lectores web para personas con discapacidad visual tratarán de leer los iconos de estas fuentes icónicas como si de texto de tratara, con el consiguiente fallo de lectura.

Actualmente se considera una mejor práctica el empleo de los llamados iconos SVG. El estándar de formato SVG significa *Scalable Vector Graphics*, es decir, gráficos vectoriales escalables, y como su nombre indica, corresponde a un formato de especificación textual en XML de gráficos vectoriales bidimensionales, esto es, un formato para describir mediante etiquetas, atributos y valores los vectores bidimensionales que conforman un determinado gráfico. Al tratarse de vectores, el gráfico resultante puede escalarse sin perder absolutamente nada de calidad, a diferencia de otros formatos gráficos NO vectoriales, como JPG, PNG, etc., que si producen pérdida de calidad. Todos los navegadores modernos están preparados para mostrar este tipo de gráficos, por lo que se recomienda su uso frente a las fuentes icónicas.

Cuando trabajamos con iconos SVG lo habitual es crear un *sprite* con los distintos iconos, de manera que tan sólo sea necesario llevar a cabo una única petición al servicio para obtener todos los iconos deseados de una vez. Luego, cuando van a usarse tan sólo tenemos que indicar exactamente que icono queremos dentro del *sprite*. De esta manera se reducen el número de peticiones al servidor y la cantidad de memoria usada, ya que al ser una descripción textual XML no ocupará mucho espacio.

Para facilitarnos la creación de estos *sprite* podemos usar herramientas online como http://icomoon.io, que permite seleccionar los iconos deseados y posteriormente descargar un sprite que crea para nosotros con dichos iconos, indicándonos además los nombres que debemos para obtener cada icono desde el sprite. Del fichero .zip descargado debemos incluir el archivo `symbol-defs.svg` en nuestro proyecto, renombrándolo normalmente a `sprite.svg`. Además deberemos hechas un vistazo también al archivo `demo.html` porque nos mostrará el nombre que debemos usar para seleccionar cada icono dentro del *sprite*.

Para incluir un icono SVG en nuestro HTML usaremos la etiqueta `<svg>`, y dentro de ella una etiqueta `<use xlink:href="ruta_sprite#nombre_icono">`. Por ejemplo:

```
<svg class="search__icon">
  <use xlink:href="img/sprite.svg#icon-magnifying-glass"></use>
</svg>
```

En el ejemplo anterior se mostrará el icono llamado `icon-magnifying-glass` situado dentro del *sprite* que se encuntra en el archivo `img/sprite.svg` indicado como path relativo al archivo HTML en el que nos encontremos.

Por otro lado podemos estilizar nuestro icono mediante CSS de una manera bastante sencilla. Por ejemplo, podemos usar `width` y `height` para especificar la anchura y altura del icono, y la propiedad `fill` para indicar el color en el que se debe mostrar el mismo (por defecto se muestran en negro). Por ejemplo:

```
.search__icon {
  width: 2rem;
  height: 2rem;
  fill: red;
}
```

No solo eso, al tratarse de etiquetas HTML, podemos manipularlas, estilizarlas o animarlas desde CSS o Javascript.

El formato svg también se puede usar como si de cualquier otro formato se tratase, es decir, por ejemplo en el atributo `src` de la etiqueta `<img>`. En ese caso no se puede modificar internamente desde CSS o Javascript.

Para estilar el borde desde CSS se usa `stroke` para el color y `stroke-width` para el ancho del borde.

La [última versión de FontAwesome]() también nos permite usar este tipo de iconos vectoriales SVG.

Máscaras

Podemos hacer que el color negro de una imagen actúe como máscara, de manera que dichas partes de la imagen se muestren con el color que haya en el fondo. Todavía no es soportado por todos los navegadores.

La propiedad `mask-image` permite indicar la imagen que actuará como máscara. En Firefox sólo vale con imágenes SVG inline y en Chrome y Safari tenemos que usar el prefijo `-webkit-mask-image`. Con `-webkit-mask-size: cover` decimos que la máscara ocupe todo el elemento (los valores son similares a los de `background-size`).

Referencias

> https://escss.blogspot.com/2013/12/propiedad-css-object-fit.html.
>
> https://escss.blogspot.com/2014/10/responsive-images-picture-srcset.html.
>
> https://css-tricks.com/clipping-masking-css/.
>
> https://escss.blogspot.com/2015/06/propiedad-css-clip-path-animada.html.
>
> https://www.smashingmagazine.com/2015/05/creating-responsive-shapes-with-clip-path/
>
> http://bashooka.com/coding/18-css-clip-path-tutorials-examples-tools/
>
> https://www.html5rocks.com/en/tutorials/masking/adobe/